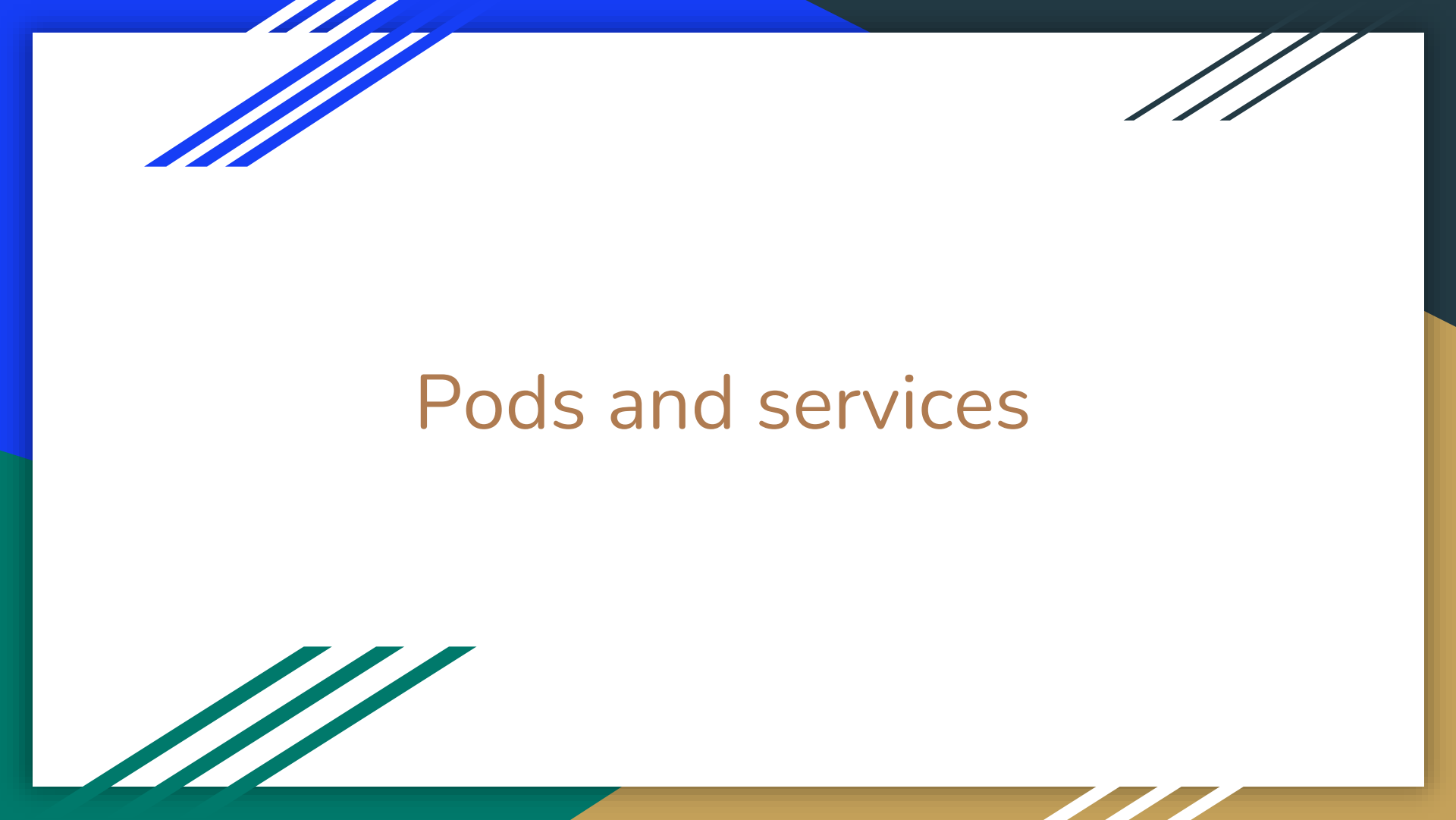




Pods and services



Pods

Pods

Pods are the smallest deployable units of computing that you can create and manage in Kubernetes.

A Pod (as in a pod of whales or pea pod) is a group of one or more containers, with shared storage and network resources, and a specification for how to run the containers. A Pod's contents are always co-located and co-scheduled, and run in a shared context. A Pod models an application-specific "logical host": it contains one or more application containers which are relatively tightly coupled. In non-cloud contexts, applications executed on the same physical or virtual machine are analogous to cloud applications executed on the same logical host.

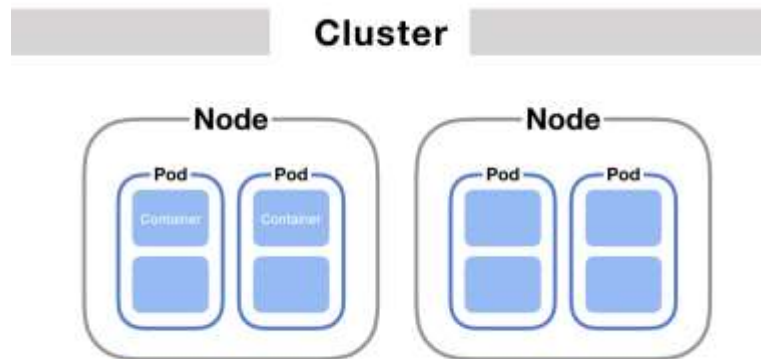
As well as application containers, a Pod can contain init containers that run during Pod startup. You can also inject ephemeral containers for debugging if your cluster offers this.

What is a Pod?

Note: While Kubernetes supports more container runtimes than just Docker, Docker is the most commonly known runtime, and it helps to describe Pods using some terminology from Docker.

The shared context of a Pod is a set of Linux namespaces, cgroups, and potentially other facets of isolation - the same things that isolate a Docker container. Within a Pod's context, the individual applications may have further sub-isolations applied.

In terms of Docker concepts, a Pod is similar to a group of Docker containers with shared namespaces and shared filesystem volumes.



Using Pods

Usually you don't need to create Pods directly, even singleton Pods. Instead, create them using workload resources such as Deployment or Job. If your Pods need to track state, consider the StatefulSet resource.

Pods in a Kubernetes cluster are used in two main ways:

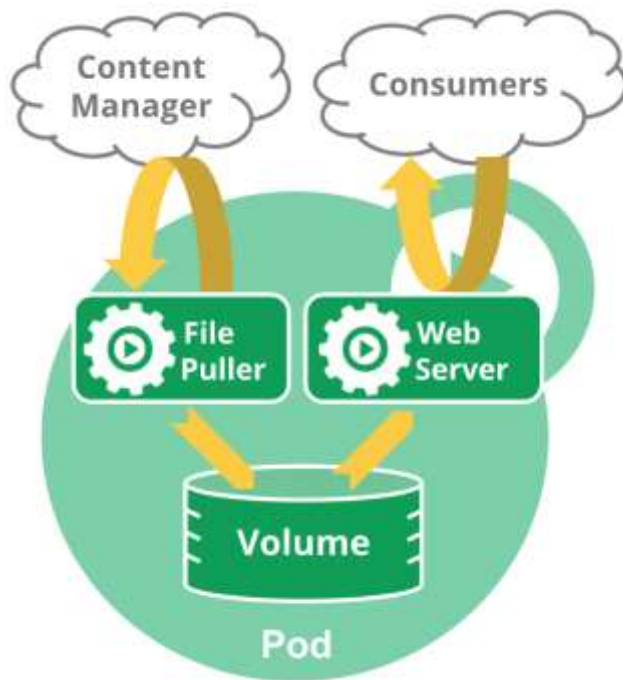
- **Pods that run a single container.** The "one-container-per-Pod" model is the most common Kubernetes use case; in this case, you can think of a Pod as a wrapper around a single container; Kubernetes manages Pods rather than managing the containers directly.
- **Pods that run multiple containers that need to work together.** A Pod can encapsulate an application composed of multiple co-located containers that are tightly coupled and need to share resources. These co-located containers form a single cohesive unit of service—for example, one container serving data stored in a shared volume to the public, while a separate sidecar container refreshes or updates those files. The Pod wraps these containers, storage resources, and an ephemeral network identity together as a single unit.

Each Pod is meant to run a single instance of a given application. If you want to scale your application horizontally (to provide more overall resources by running more instances), you should use multiple Pods, one for each instance. In Kubernetes, this is typically referred to as replication. Replicated Pods are usually created and managed as a group by a workload resource and its controller.

How Pods manage multiple containers

Pods are designed to support multiple cooperating processes (as containers) that form a cohesive unit of service. The containers in a Pod are automatically co-located and co-scheduled on the same physical or virtual machine in the cluster. The containers can share resources and dependencies, communicate with one another, and coordinate when and how they are terminated.

For example, you might have a container that acts as a web server for files in a shared volume, and a separate "sidecar" container that updates those files from a remote source, as in the following diagram:



How Pods manage multiple containers

Some Pods have init containers as well as app containers. Init containers run and complete before the app containers are started.

Pods natively provide two kinds of shared resources for their constituent containers: networking and storage.

Working with Pods

You'll rarely create individual Pods directly in Kubernetes—even singleton Pods. This is because Pods are designed as relatively ephemeral, disposable entities. When a Pod gets created (directly by you, or indirectly by a controller), the new Pod is scheduled to run on a Node in your cluster. The Pod remains on that node until the Pod finishes execution, the Pod object is deleted, the Pod is evicted for lack of resources, or the node fails.

When you create the manifest for a Pod object, make sure the name specified is a valid DNS subdomain name.

Pods and controllers

You can use workload resources to create and manage multiple Pods for you. A controller for the resource handles replication and rollout and automatic healing in case of Pod failure. For example, if a Node fails, a controller notices that Pods on that Node have stopped working and creates a replacement Pod. The scheduler places the replacement Pod onto a healthy Node.

Here are some examples of workload resources that manage one or more Pods:

- Deployment
- StatefulSet
- DaemonSet

Pod templates

Controllers for workload resources create Pods from a pod template and manage those Pods on your behalf.

PodTemplates are specifications for creating Pods, and are included in workload resources such as Deployments, Jobs, and DaemonSets.

Each controller for a workload resource uses the PodTemplate inside the workload object to make actual Pods. The PodTemplate is part of the desired state of whatever workload resource you used to run your app.

The sample below is a manifest for a simple Job with a template that starts one container. The container in that Pod prints a message then pauses.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello
spec:
  template:
    # This is the pod template
    spec:
      containers:
        - name: hello
          image: busybox
          command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 3600']
          restartPolicy: OnFailure
    # The pod template ends here
```

Pod templates

Modifying the pod template or switching to a new pod template has no direct effect on the Pods that already exist. If you change the pod template for a workload resource, that resource needs to create replacement Pods that use the updated template.

For example, the StatefulSet controller ensures that the running Pods match the current pod template for each StatefulSet object. If you edit the StatefulSet to change its pod template, the StatefulSet starts to create new Pods based on the updated template. Eventually, all of the old Pods are replaced with new Pods, and the update is complete.

Each workload resource implements its own rules for handling changes to the Pod template. On Nodes, the kubelet does not directly observe or manage any of the details around pod templates and updates; those details are abstracted away. That abstraction and separation of concerns simplifies system semantics, and makes it feasible to extend the cluster's behavior without changing existing code.

Pod update and replacement

As mentioned in the previous section, when the Pod template for a workload resource is changed, the controller creates new Pods based on the updated template instead of updating or patching the existing Pods.

Kubernetes doesn't prevent you from managing Pods directly. It is possible to update some fields of a running Pod, in place. However, Pod update operations like `patch`, and `replace` have some limitations:

- Most of the metadata about a Pod is immutable. For example, you cannot change the `namespace`, `name`, `uid`, or `creationTimestamp` fields; the `generation` field is unique. It only accepts updates that increment the field's current value.
- If the `metadata.deletionTimestamp` is set, no new entry can be added to the `metadata.finalizers` list.
- Pod updates may not change fields other than `spec.containers[*].image`, `spec.initContainers[*].image`, `spec.activeDeadlineSeconds` or `spec.tolerations`. For `spec.tolerations`, you can only add new entries.
- When updating the `spec.activeDeadlineSeconds` field, two types of updates are allowed:
 1. setting the unassigned field to a positive number;
 2. updating the field from a positive number to a smaller, non-negative number.

Resource sharing and communication

Pods enable data sharing and communication among their constituent containers.

Storage in Pods

A Pod can specify a set of shared storage volumes. All containers in the Pod can access the shared volumes, allowing those containers to share data. Volumes also allow persistent data in a Pod to survive in case one of the containers within needs to be restarted.

Resource sharing and communication

Pod networking

Each Pod is assigned a unique IP address for each address family. Every container in a Pod shares the network namespace, including the IP address and network ports. Inside a Pod (and only then), the containers that belong to the Pod can communicate with one another using localhost. When containers in a Pod communicate with entities outside the Pod, they must coordinate how they use the shared network resources (such as ports). Within a Pod, containers share an IP address and port space, and can find each other via localhost. The containers in a Pod can also communicate with each other using standard inter-process communications like SystemV semaphores or POSIX shared memory. Containers in different Pods have distinct IP addresses and can not communicate by IPC without special configuration. Containers that want to interact with a container running in a different Pod can use IP networking to communicate.

Containers within the Pod see the system hostname as being the same as the configured name for the Pod.

Privileged mode for containers

Any container in a Pod can enable privileged mode, using the privileged flag on the security context of the container spec. This is useful for containers that want to use operating system administrative capabilities such as manipulating the network stack or accessing hardware devices. Processes within a privileged container get almost the same privileges that are available to processes outside a container.

Static Pods

Static Pods are managed directly by the kubelet daemon on a specific node, without the API server observing them. Whereas most Pods are managed by the control plane (for example, a Deployment), for static Pods, the kubelet directly supervises each static Pod (and restarts it if it fails).

Static Pods are always bound to one Kubelet on a specific node. The main use for static Pods is to run a self-hosted control plane: in other words, using the kubelet to supervise the individual control plane components.

The kubelet automatically tries to create a mirror Pod on the Kubernetes API server for each static Pod. This means that the Pods running on a node are visible on the API server, but cannot be controlled from there.



Pod Lifecycle

Pod phase

A Pod's `status` field is a `PodStatus` object, which has a `phase` field.

The phase of a Pod is a simple, high-level summary of where the Pod is in its lifecycle. The phase is not intended to be a comprehensive rollup of observations of container or Pod state, nor is it intended to be a comprehensive state machine.

The number and meanings of Pod phase values are tightly guarded. Other than what is documented here, nothing should be assumed about Pods that have a given `phase` value. If a node dies or is disconnected from the rest of the cluster, Kubernetes applies a policy for setting the `phase` of all Pods on the lost node to `Failed`.

Here are the possible values for `phase`:

Value	Description
<code>Pending</code>	The Pod has been accepted by the Kubernetes cluster, but one or more of the containers has not been set up and made ready to run. This includes time a Pod spends waiting to be scheduled as well as the time spent downloading container images over the network.
<code>Running</code>	The Pod has been bound to a node, and all of the containers have been created. At least one container is still running, or is in the process of starting or restarting.
<code>Succeeded</code>	All containers in the Pod have terminated in success, and will not be restarted.
<code>Failed</code>	All containers in the Pod have terminated, and at least one container has terminated in failure. That is, the container either exited with non-zero status or was terminated by the system.
<code>Unknown</code>	For some reason the state of the Pod could not be obtained. This phase typically occurs due to an error in communicating with the node where the Pod should be running.

Container states

As well as the phase of the Pod overall, Kubernetes tracks the state of each container inside a Pod. You can use container lifecycle hooks to trigger events to run at certain points in a container's lifecycle.

Once the scheduler assigns a Pod to a Node, the kubelet starts creating containers for that Pod using a container runtime. There are three possible container states: Waiting, Running, and Terminated.

To check the state of a Pod's containers, you can use `kubectl describe pod <name-of-pod>`. The output shows the state for each container within that Pod.

Each state has a specific meaning:

Waiting

If a container is not in either the Running or Terminated state, it is Waiting. A container in the Waiting state is still running the operations it requires in order to complete start up: for example, pulling the container image from a container image registry, or applying Secret data. When you use `kubectl` to query a Pod with a container that is Waiting, you also see a Reason field to summarize why the container is in that state.

Container states

Running

The `Running` status indicates that a container is executing without issues. If there was a `postStart` hook configured, it has already executed and finished. When you use `kubectl` to query a Pod with a container that is `Running`, you also see information about when the container entered the `Running` state.

Terminated

A container in the `Terminated` state began execution and then either ran to completion or failed for some reason. When you use `kubectl` to query a Pod with a container that is `Terminated`, you see a reason, an exit code, and the start and finish time for that container's period of execution.

If a container has a `preStop` hook configured, that runs before the container enters the `Terminated` state.

Container restart policy

The spec of a Pod has a `restartPolicy` field with possible values `Always`, `OnFailure`, and `Never`. The default value is `Always`.

The `restartPolicy` applies to all containers in the Pod. `restartPolicy` only refers to restarts of the containers by the kubelet on the same node. After containers in a Pod exit, the kubelet restarts them with an exponential back-off delay (10s, 20s, 40s, ...), that is capped at five minutes. Once a container has executed for 10 minutes without any problems, the kubelet resets the restart backoff timer for that container.

Pod conditions

A Pod has a PodStatus, which has an array of PodConditions through which the Pod has or has not passed:

- **PodScheduled**: the Pod has been scheduled to a node.
- **ContainersReady**: all containers in the Pod are ready.
- **Initialized**: all init containers have started successfully.
- **Ready**: the Pod is able to serve requests and should be added to the load balancing pools of all matching Services.

Field name	Description
<code>type</code>	Name of this Pod condition.
<code>status</code>	Indicates whether that condition is applicable, with possible values "True", "False", or "Unknown".
<code>lastProbeTime</code>	Timestamp of when the Pod condition was last probed.
<code>lastTransitionTime</code>	Timestamp for when the Pod last transitioned from one status to another.
<code>reason</code>	Machine-readable, UpperCamelCase text indicating the reason for the condition's last transition.
<code>message</code>	Human-readable message indicating details about the last status transition.

Container probes

Each probe has one of three results:

- `Success`: The container passed the diagnostic.
- `Failure`: The container failed the diagnostic.
- `Unknown`: The diagnostic failed, so no action should be taken.

Container probes

The kubelet can optionally perform and react to three kinds of probes on running containers:

- `livenessProbe`: Indicates whether the container is running. If the liveness probe fails, the kubelet kills the container, and the container is subjected to its restart policy. If a Container does not provide a liveness probe, the default state is Success.
- `readinessProbe`: Indicates whether the container is ready to respond to requests. If the readiness probe fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod. The default state of readiness before the initial delay is Failure. If a Container does not provide a readiness probe, the default state is Success.
- `startupProbe`: Indicates whether the application within the container is started. All other probes are disabled if a startup probe is provided, until it succeeds. If the startup probe fails, the kubelet kills the container, and the container is subjected to its restart policy. If a Container does not provide a startup probe, the default state is Success.

When should you use a liveness probe?

FEATURE STATE: Kubernetes v1.0 [stable]

If the process in your container is able to crash on its own whenever it encounters an issue or becomes unhealthy, you do not necessarily need a liveness probe; the kubelet will automatically perform the correct action in accordance with the Pod's `restartPolicy`.

If you'd like your container to be killed and restarted if a probe fails, then specify a liveness probe, and specify a `restartPolicy` of `Always` or `OnFailure`.

When should you use a readiness probe?

FEATURE STATE: Kubernetes v1.0 [stable]

If you'd like to start sending traffic to a Pod only when a probe succeeds, specify a readiness probe. In this case, the readiness probe might be the same as the liveness probe, but the existence of the readiness probe in the spec means that the Pod will start without receiving any traffic and only start receiving traffic after the probe starts succeeding. If your container needs to work on loading large data, configuration files, or migrations during startup, specify a readiness probe.

If you want your container to be able to take itself down for maintenance, you can specify a readiness probe that checks an endpoint specific to readiness that is different from the liveness probe.

When should you use a startup probe?

FEATURE STATE: Kubernetes v1.20 [stable]

Startup probes are useful for Pods that have containers that take a long time to come into service. Rather than set a long liveness interval, you can configure a separate configuration for probing the container as it starts up, allowing a time longer than the liveness interval would allow.

If your container usually starts in more than `initialDelaySeconds + failureThreshold × periodSeconds`, you should specify a startup probe that checks the same endpoint as the liveness probe. The default for `periodSeconds` is 10s. You should then set its `failureThreshold` high enough to allow the container to start, without changing the default values of the liveness probe. This helps to protect against deadlocks.

Termination of Pods

Because Pods represent processes running on nodes in the cluster, it is important to allow those processes to gracefully terminate when they are no longer needed (rather than being abruptly stopped with a KILL signal and having no chance to clean up).

The design aim is for you to be able to request deletion and know when processes terminate, but also be able to ensure that deletes eventually complete. When you request deletion of a Pod, the cluster records and tracks the intended grace period before the Pod is allowed to be forcefully killed. With that forceful shutdown tracking in place, the kubelet attempts graceful shutdown.

Typically, the container runtime sends a TERM signal to the main process in each container. Many container runtimes respect the STOPSIGNAL value defined in the container image and send this instead of TERM. Once the grace period has expired, the KILL signal is sent to any remaining processes, and the Pod is then deleted from the API Server. If the kubelet or the container runtime's management service is restarted while waiting for processes to terminate, the cluster retries from the start including the full original grace period.

Termination of Pods

An example flow:

1. You use the `kubectl` tool to manually delete a specific Pod, with the default grace period (30 seconds).
2. The Pod in the API server is updated with the time beyond which the Pod is considered "dead" along with the grace period. If you use `kubectl describe` to check on the Pod you're deleting, that Pod shows up as "Terminating". On the node where the Pod is running: as soon as the kubelet sees that a Pod has been marked as terminating (a graceful shutdown duration has been set), the kubelet begins the local Pod shutdown process.
 1. If one of the Pod's containers has defined a `preStop` hook, the kubelet runs that hook inside of the container. If the `preStop` hook is still running after the grace period expires, the kubelet requests a small, one-off grace period extension of 2 seconds.
 2. The kubelet triggers the container runtime to send a `TERM` signal to process 1 inside each container.
3. At the same time as the kubelet is starting graceful shutdown, the control plane removes that shutting-down Pod from Endpoints (and, if enabled, `EndpointSlice`) objects where these represent a [Service](#) with a configured [selector](#). [ReplicaSets](#) and other workload resources no longer treat the shutting-down Pod as a valid, in-service replica. Pods that shut down slowly cannot continue to serve traffic as load balancers (like the service proxy) remove the Pod from the list of endpoints as soon as the termination grace period begins.

Termination of Pods

4. When the grace period expires, the kubelet triggers forcible shutdown. The container runtime sends SIGKILL to any processes still running in any container in the Pod. The kubelet also cleans up a hidden pause container if that container runtime uses one.
5. The kubelet triggers forcible removal of Pod object from the API server, by setting grace period to 0 (immediate deletion).
6. The API server deletes the Pod's API object, which is then no longer visible from any client.

Forced Pod termination

By default, all deletes are graceful within 30 seconds. The `kubectl delete` command supports the `--grace-period=<seconds>` option which allows you to override the default and specify your own value.

Setting the grace period to 0 forcibly and immediately deletes the Pod from the API server. If the pod was still running on a node, that forcible deletion triggers the kubelet to begin immediate cleanup.

When a force deletion is performed, the API server does not wait for confirmation from the kubelet that the Pod has been terminated on the node it was running on. It removes the Pod in the API immediately so a new Pod can be created with the same name. On the node, Pods that are set to terminate immediately will still be given a small grace period before being force killed.

Garbage collection of failed Pods

For failed Pods, the API objects remain in the cluster's API until a human or controller process explicitly removes them.

The control plane cleans up terminated Pods (with a phase of Succeeded or Failed), when the number of Pods exceeds the configured threshold (determined by `terminated-pod-gc-threshold` in the `kube-controller-manager`). This avoids a resource leak as Pods are created and terminated over time.



Service

Service

An abstract way to expose an application running on a set of Pods as a network service.

With Kubernetes you don't need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

Service resources

In Kubernetes, a Service is an abstraction which defines a logical set of Pods and a policy by which to access them (sometimes this pattern is called a micro-service). The set of Pods targeted by a Service is usually determined by a selector. For example, consider a stateless image-processing backend which is running with 3 replicas. Those replicas are fungible—frontends do not care which backend they use. While the actual Pods that compose the backend set may change, the frontend clients should not need to be aware of that, nor should they need to keep track of the set of backends themselves. The Service abstraction enables this decoupling.

Cloud-native service discovery

If you're able to use Kubernetes APIs for service discovery in your application, you can query the API server for Endpoints, that get updated whenever the set of Pods in a Service changes.

For non-native applications, Kubernetes offers ways to place a network port or load balancer in between your application and the backend Pods.

Defining a Service

A Service in Kubernetes is a REST object, similar to a Pod. Like all of the REST objects, you can POST a Service definition to the API server to create a new instance. The name of a Service object must be a valid DNS label name.

For example, suppose you have a set of Pods that each listen on TCP port 9376 and carry a label `app=MyApp`:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Defining a Service

This specification creates a new Service object named "my-service", which targets TCP port 9376 on any Pod with the app=MyApp label.

Kubernetes assigns this Service an IP address (sometimes called the "cluster IP"), which is used by the Service proxies. The controller for the Service selector continuously scans for Pods that match its selector, and then POSTs any updates to an Endpoint object also named "my-service".

Port definitions in Pods have names, and you can reference these names in the targetPort attribute of a Service. This works even if there is a mixture of Pods in the Service using a single configured name, with the same network protocol available via different port numbers. This offers a lot of flexibility for deploying and evolving your Services. For example, you can change the port numbers that Pods expose in the next version of your backend software, without breaking clients.

The default protocol for Services is TCP. As many Services need to expose more than one port, Kubernetes supports multiple port definitions on a Service object. Each port definition can have the same protocol, or a different one.

Services without selectors

Services most commonly abstract access to Kubernetes Pods, but they can also abstract other kinds of backends. For example:

- You want to have an external database cluster in production, but in your test environment you use your own databases.
- You want to point your Service to a Service in a different Namespace or on another cluster.
- You are migrating a workload to Kubernetes. While evaluating the approach, you run only a portion of your backends in Kubernetes.

In any of these scenarios you can define a Service without a Pod selector. For example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Services without selectors

Because this Service has no selector, the corresponding Endpoints object is not created automatically. You can manually map the Service to the network address and port where it's running, by adding an Endpoints object manually:

The name of the Endpoints object must be a valid DNS subdomain name.

Accessing a Service without a selector works the same as if it had a selector. In the example above, traffic is routed to the single endpoint defined in the YAML: 192.0.2.42:9376 (TCP).

An ExternalName Service is a special case of Service that does not have selectors and uses DNS names instead.

```
apiVersion: v1
kind: Endpoints
metadata:
  name: my-service
subsets:
  - addresses:
      - ip: 192.0.2.42
    ports:
      - port: 9376
```

EndpointSlices

FEATURE STATE: `Kubernetes v1.17 [beta]`

EndpointSlices are an API resource that can provide a more scalable alternative to Endpoints. Although conceptually quite similar to Endpoints, EndpointSlices allow for distributing network endpoints across multiple resources. By default, an EndpointSlice is considered "full" once it reaches 100 endpoints, at which point additional EndpointSlices will be created to store any additional endpoints.

EndpointSlices provide additional attributes and functionality which is described in detail in [EndpointSlices](#).

Application protocol

FEATURE STATE: Kubernetes v1.20 [stable]

The appProtocol field provides a way to specify an application protocol for each Service port. The value of this field is mirrored by the corresponding Endpoints and EndpointSlice objects.

This field follows standard Kubernetes label syntax. Values should either be IANA standard service names or domain prefixed names such as mycompany.com/my-custom-protocol.

Virtual IPs and service proxies

Every node in a Kubernetes cluster runs a kube-proxy. kube-proxy is responsible for implementing a form of virtual IP for Services of type other than ExternalName.

Why not use round-robin DNS?

A question that pops up every now and then is why Kubernetes relies on proxying to forward inbound traffic to backends. What about other approaches? For example, would it be possible to configure DNS records that have multiple A values (or AAAA for IPv6), and rely on round-robin name resolution?

There are a few reasons for using proxying for Services:

- There is a long history of DNS implementations not respecting record TTLs, and caching the results of name lookups after they should have expired.
- Some apps do DNS lookups only once and cache the results indefinitely.
- Even if apps and libraries did proper re-resolution, the low or zero TTLs on the DNS records could impose a high load on DNS that then becomes difficult to manage.

User space proxy mode

In this mode, kube-proxy watches the Kubernetes control plane for the addition and removal of Service and Endpoint objects. For each Service it opens a port (randomly chosen) on the local node. Any connections to this "proxy port" are proxied to one of the Service's backend Pods (as reported via Endpoints). kube-proxy takes the SessionAffinity setting of the Service into account when deciding which backend Pod to use.

Lastly, the user-space proxy installs iptables rules which capture traffic to the Service's clusterIP (which is virtual) and port. The rules redirect that traffic to the proxy port which proxies the backend Pod.

By default, kube-proxy in userspace mode chooses a backend via a round-robin algorithm.

Multi-Port Services

For some Services, you need to expose more than one port.

Kubernetes lets you configure multiple port definitions on a Service object. When using multiple ports for a Service, you must give all of your ports names so that these are unambiguous. For example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
    - name: https
      protocol: TCP
      port: 443
      targetPort: 9377
```

Choosing your own IP address

You can specify your own cluster IP address as part of a `Service` creation request. To do this, set the `.spec.clusterIP` field. For example, if you already have an existing DNS entry that you wish to reuse, or legacy systems that are configured for a specific IP address and difficult to re-configure.

The IP address that you choose must be a valid IPv4 or IPv6 address from within the `service-cluster-ip-range` CIDR range that is configured for the API server. If you try to create a `Service` with an invalid `clusterIP` address value, the API server will return a 422 HTTP status code to indicate that there's a problem.

Discovering services

Kubernetes supports 2 primary modes of finding a Service - environment variables and DNS.

Environment variables

When a Pod is run on a Node, the kubelet adds a set of environment variables for each active Service. It supports both Docker links compatible variables and simpler `{SVCNAME}_SERVICE_HOST` and `{SVCNAME}_SERVICE_PORT` variables, where the Service name is upper-cased and dashes are converted to underscores.

For example, the Service `redis-master` which exposes TCP port 6379 and has been allocated cluster IP address 10.0.0.11, produces the following environment variables:

```
REDIS_MASTER_SERVICE_HOST=10.0.0.11
REDIS_MASTER_SERVICE_PORT=6379
REDIS_MASTER_PORT=tcp://10.0.0.11:6379
REDIS_MASTER_PORT_6379_TCP=tcp://10.0.0.11:6379
REDIS_MASTER_PORT_6379_TCP_PROTO=tcp
REDIS_MASTER_PORT_6379_TCP_PORT=6379
REDIS_MASTER_PORT_6379_TCP_ADDR=10.0.0.11
```

Publishing Services (ServiceTypes)

For some parts of your application (for example, frontends) you may want to expose a Service onto an external IP address, that's outside of your cluster.

Kubernetes `ServiceTypes` allow you to specify what kind of Service you want. The default is `ClusterIP`.

Type values and their behaviors are:

- `ClusterIP`: Exposes the Service on a cluster-internal IP. Choosing this value makes the Service only reachable from within the cluster. This is the default `ServiceType`.
- `NodePort`: Exposes the Service on each Node's IP at a static port (the `NodePort`). A `ClusterIP` Service, to which the `NodePort` Service routes, is automatically created. You'll be able to contact the `NodePort` Service, from outside the cluster, by requesting `<NodeIP>:<NodePort>`.
- `LoadBalancer`: Exposes the Service externally using a cloud provider's load balancer. `NodePort` and `ClusterIP` Services, to which the external load balancer routes, are automatically created.
- `ExternalName`: Maps the Service to the contents of the `externalName` field (e.g. `foo.bar.example.com`), by returning a CNAME record with its value. No proxying of any kind is set up.

Type NodePort

If you set the type field to NodePort, the Kubernetes control plane allocates a port from a range specified by `--service-node-port-range` flag (default: 30000-32767). Each node proxies that port (the same port number on every Node) into your Service. Your Service reports the allocated port in its `.spec.ports[*].nodePort` field.

If you want to specify particular IP(s) to proxy the port, you can set the `--nodeport-addresses` flag in kube-proxy to particular IP block(s); this is supported since Kubernetes v1.10. This flag takes a comma-delimited list of IP blocks (e.g. 10.0.0.0/8, 192.0.2.0/25) to specify IP address ranges that kube-proxy should consider as local to this node.

For example, if you start kube-proxy with the `--nodeport-addresses=127.0.0.0/8` flag, kube-proxy only selects the loopback interface for NodePort Services. The default for `--nodeport-addresses` is an empty list. This means that kube-proxy should consider all available network interfaces for NodePort. (That's also compatible with earlier Kubernetes releases).

If you want a specific port number, you can specify a value in the `nodePort` field. The control plane will either allocate you that port or report that the API transaction failed. This means that you need to take care of possible port collisions yourself. You also have to use a valid port number, one that's inside the range configured for NodePort use.

Type NodePort

Using a NodePort gives you the freedom to set up your own load balancing solution, to configure environments that are not fully supported by Kubernetes, or even to just expose one or more nodes' IPs directly.

Note that this Service is visible as `<NodeIP>:spec.ports[*].nodePort` and `.spec.clusterIP:spec.ports[*].port`. (If the `--nodeport-addresses` flag in kube-proxy is set, would be filtered NodeIP(s).)

For example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: MyApp
  ports:
    # By default and for convenience, the "targetPort" is set to the same value as the "port" field.
    - port: 88
      targetPort: 88
    # Optional field
    # By default and for convenience, the Kubernetes control plane will allocate a port from a range (default: 30000-32767)
    nodePort: 30007
```

Type LoadBalancer

On cloud providers which support external load balancers, setting the `type` field to `LoadBalancer` provisions a load balancer for your Service. The actual creation of the load balancer happens asynchronously, and information about the provisioned balancer is published in the Service's `.status.loadBalancer` field. For example:

Traffic from the external load balancer is directed at the backend Pods. The cloud provider decides how it is load balanced.

Some cloud providers allow you to specify the `loadBalancerIP`. In those cases, the load-balancer is created with the user-specified `loadBalancerIP`. If the `loadBalancerIP` field is not specified, the `loadBalancer` is set up with an ephemeral IP address. If you specify a `loadBalancerIP` but your cloud provider does not support the feature, the `loadBalancerIP` field that you set is ignored.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  clusterIP: 10.0.171.239
  type: LoadBalancer
status:
  loadBalancer:
    ingress:
      - ip: 192.0.2.127
```

Type ExternalName

Services of type ExternalName map a Service to a DNS name, not to a typical selector such as my-service or cassandra. You specify these Services with the spec.externalName parameter.

This Service definition, for example, maps the my-service Service in the prod namespace to my.database.example.com:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

Type ExternalName

When looking up the host `my-service.prod.svc.cluster.local`, the cluster DNS Service returns a `CNAME` record with the value `my.database.example.com`. Accessing `my-service` works in the same way as other Services but with the crucial difference that redirection happens at the DNS level rather than via proxying or forwarding. Should you later decide to move your database into your cluster, you can start its Pods, add appropriate selectors or endpoints, and change the Service's type.

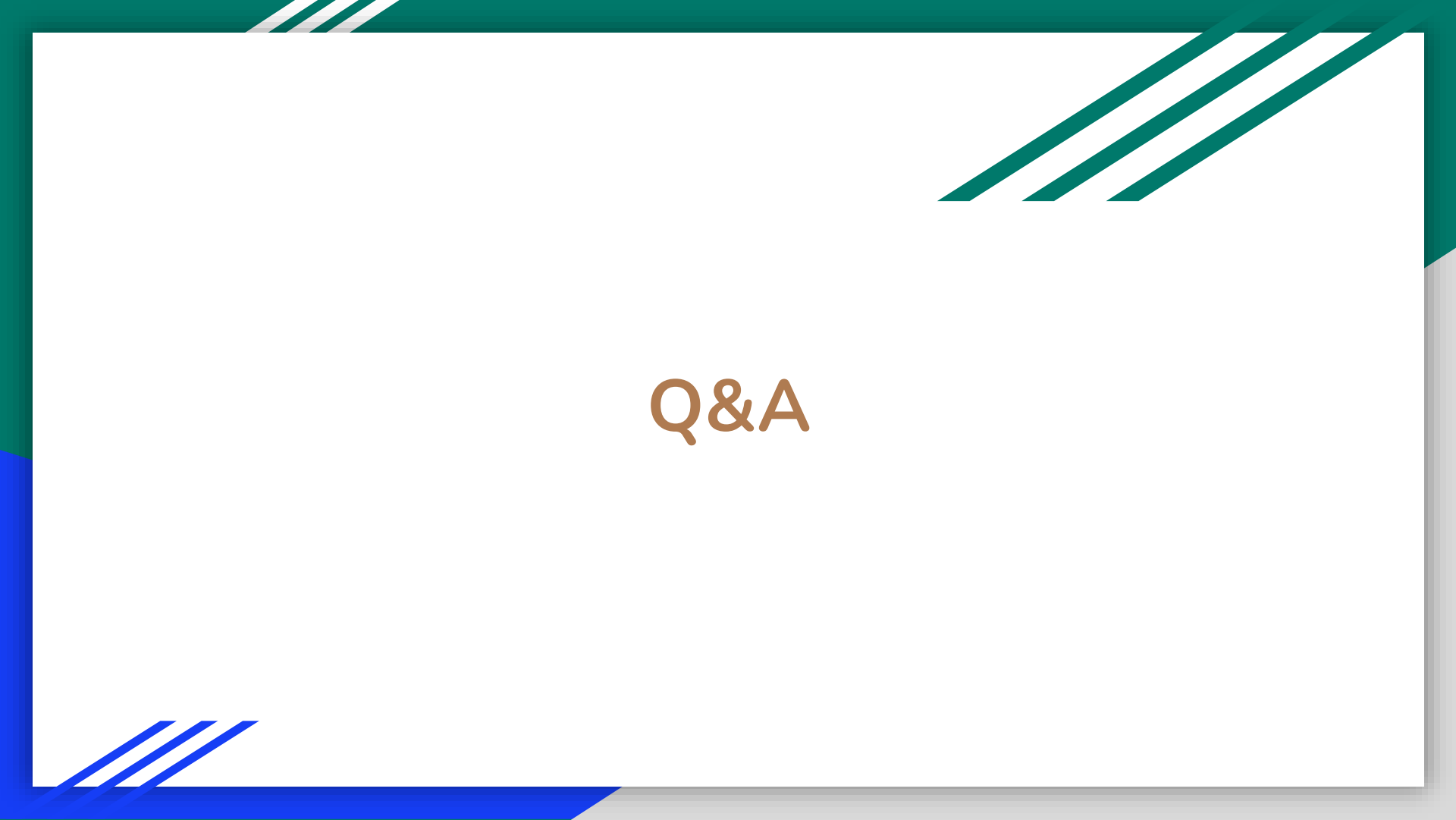
```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

External IPs

If there are external IPs that route to one or more cluster nodes, Kubernetes Services can be exposed on those externalIPs. Traffic that ingresses into the cluster with the `external IP` (as destination IP), on the Service port, will be routed to one of the Service endpoints. `externalIPs` are not managed by Kubernetes and are the responsibility of the cluster administrator.

In the Service spec, `externalIPs` can be specified along with any of the `ServiceTypes`. In the example below, "my-service" can be accessed by clients on "80.11.12.10:80" (`externalIP:port`)

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
  externalIPs:
    - 80.11.12.10
```



Q&A